

IT INCIDENT COPILOT

Comprehensive Enterprise System & Feature Documentation

Complete Guide to Features, Architecture, Database Persistence & Security

SYSTEM SPECIFICATIONS SUMMARY

- Frontend Stack: Vite, React 19, TailwindCSS, Lucide Icons
- Backend Stack: ASP.NET Core 9/10 C#, EF Core, BCrypt, JWT
- Primary Database: PostgreSQL (Port 5432 / DBeaver Sync)
- Fallback Engine: SQLite (it_copilot.db) & Global Cloud KV Bin
- Deployment Targets: Vercel Cloud & Local Host (Port 5000 / 5173)

Generated: Sunday, September 13, 2026

1. EXECUTIVE SUMMARY & SYSTEM OVERVIEW

1.1 Purpose & Core Mission

The IT Incident Copilot is an enterprise-grade ticketing and diagnostic automation platform designed to make support management simple, fast, and accessible for everyone. It bridges non-technical staff with Tier-2 systems engineers through intelligent automation, real-time PC telemetry diagnostics, and simple English communication.

1.2 Key Architectural Pillars

1. Pure & Simple Ticketing UI

Streamlined ticket lifecycle management written in plain, accessible English with full multi-language translation support.

2. Automated Device Telemetry

Extracts real-time PC performance metrics (CPU, RAM, Disk, Network Latency, Event Logs) to pinpoint root causes automatically.

3. Multi-Engine Persistence

Connects to PostgreSQL for DBeaver live inspection, SQLite for zero-config local runs, and Global Cloud KV Store for Vercel deployments.

4. Enterprise Security & RBAC

BCrypt password hashing, JWT authorization, email duplicate enforcement, and auto-fill credential protection.

2. SYSTEM ARCHITECTURE & DATABASE SYNC

The system implements a triple-redundant data persistence architecture ensuring reliability across local development, SQL tools (DBeaver), and Vercel cloud deployments:

PERSISTENCE TIER ARCHITECTURE

1. Primary Database (PostgreSQL - Port 5432):
 - Configured in backend/appsettings.json (Host=localhost;Database=it_copilot).
 - Directly synced with DBeaver SQL IDE for live table inspection & auditing.
2. Local Zero-Config Fallback (SQLite - it_copilot.db):
 - Automatic fallback if PostgreSQL server is offline during local development.
3. Global Cloud Bin Sync (Vercel Cloud Deployment):
 - Syncs registered accounts across ALL devices globally when deployed on Vercel.

3. CORE FEATURES & MODULE SPECIFICATIONS

3.1 User Authentication & Role Control (RBAC)

- BCrypt Password Hashing & JWT Token Issuance.
- Strict Email Uniqueness Check: Prevents creating duplicate accounts with the same email address both on backend API and frontend storage.
- Auto-Fill & URL Protection: Credentials are never pre-filled by browser autofill and are strictly sent via POST body payload—never in website URLs.
- Persona Switching: Toggle between Administrator (Lead System Admin), Technician (Senior Systems Engineer), and Employee (Staff Member).

3.2 Simplified Employee Support Portal

- Self-Service Ticket Submission: Staff can report IT issues using plain, simple English.
- Automated PC Telemetry Attachment: Extracts computer hostname, IP address, and MAC address automatically.
- Attachment Support: Attach screenshot logs or error dumps directly to support tickets.
- My Support Tickets: Employees view and track only their own reported incidents.

3.3 Technician Incident Workstation & AI Copilot

- Real-Time Telemetry Diagnostic Engine: Monitors CPU, Memory Heap, Disk Utilization, Network Latency, and Event Logs.
- Deterministic Rule Engine: Evaluates 5 diagnostic rules against live telemetry metrics.
- Automated Playbook Recommendations: Suggests automated remediation procedures for rapid resolution.
- State Machine Audit Trail: Records every status transition (NEW -> INVESTIGATING -> RESOLVED -> CLOSED) with actor tracking.

3.4 Manager Analytics & Metrics Dashboard

- Resolution SLA Metrics: Tracks ticket resolution speed and SLA compliance in real time.
- Category Distribution Breakdown: Visualizes incidents by category (EndUser, Infrastructure, Identity, Database).
- User Control Center: Admins can elevate user roles (Employee -> Technician -> Administrator).

4. BACKEND API ENDPOINT REFERENCE

POST	/api/auth/register	Registers new user account with BCrypt hash & email uniqueness check.
POST	/api/auth/login	Authenticates credentials and returns JWT access & refresh tokens.
GET	/api/auth/users	Lists all system accounts for Admin Control Center.
PUT	/api/auth/users/{id}/role	Updates user role (EMPLOYEE, TECHNICIAN, ADMINISTRATOR).
POST	/api/auth/reset-password	Resets account password with validation.
GET	/api/incidents	Fetches enterprise support tickets.
POST	/api/incidents	Creates new ticket with telemetry data.
PUT	/api/incidents/{id}/status	Modifies incident lifecycle state with audit logging.

5. SETUP, DBEAVER SYNC & DEPLOYMENT GUIDE

5.1 Running Locally & DBeaver Database Connection

To inspect live database records using DBeaver or run the project locally:

- 1 Start PostgreSQL Server**
Ensure PostgreSQL is running locally on port 5432.
- 2 Configure appsettings.json**
backend/appsettings.json sets "DefaultConnection": "Host=localhost;Database=it_copilot;Username=postgres".
- 3 Run C# Backend API**
Execute "dotnet run" inside backend/ folder. The API initializes tables and seeds default users.
- 4 Start Frontend Web App**
Execute "npm run dev" in root workspace to launch Vite dev server at <http://localhost:5173/>.
- 5 Connect DBeaver**
Open DBeaver -> Connect PostgreSQL -> Host: localhost, Database: it_copilot. Open "Users" table and press F5 to refresh.

5.2 Deploying to Vercel Cloud

The repository is connected to Vercel. Pushing to origin/main automatically triggers production deployments. The built-in Global Cloud KV Store ensures all users created on Vercel persist globally across all devices.